

Problem Statement

Application is running fine in development environment but not working in QA environment.

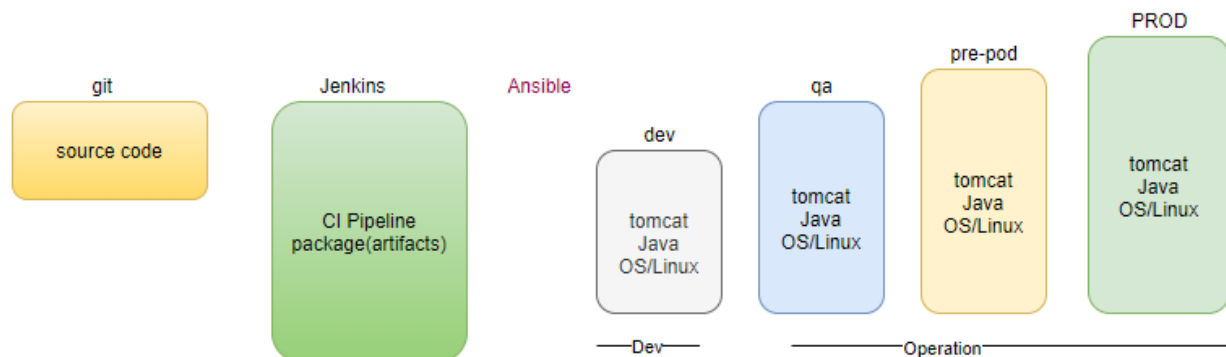
Reason:

- Different versions
- Dependencies

How to ship application in to different environment,

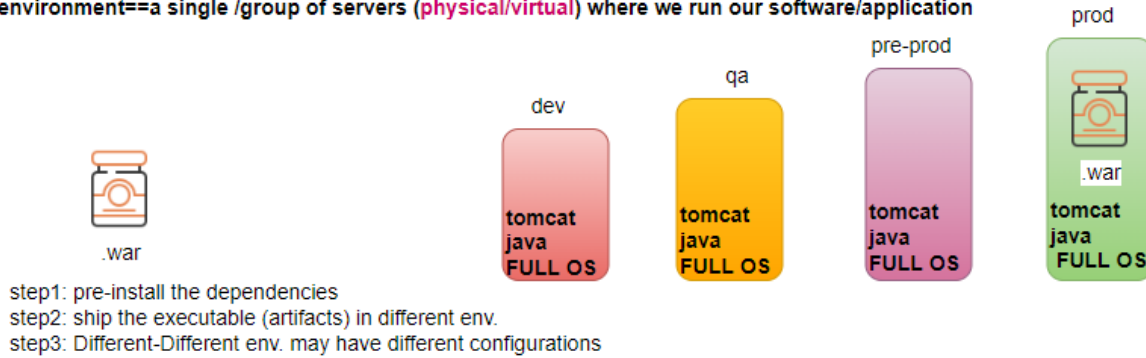
Environments (o/s, java, and tomcat) must be ready before deploying the artifacts (.war), we just move artifacts from one environment to another environments.

Configuration Management tool (Ansible) help us to environment setup and deploy application in different environments.



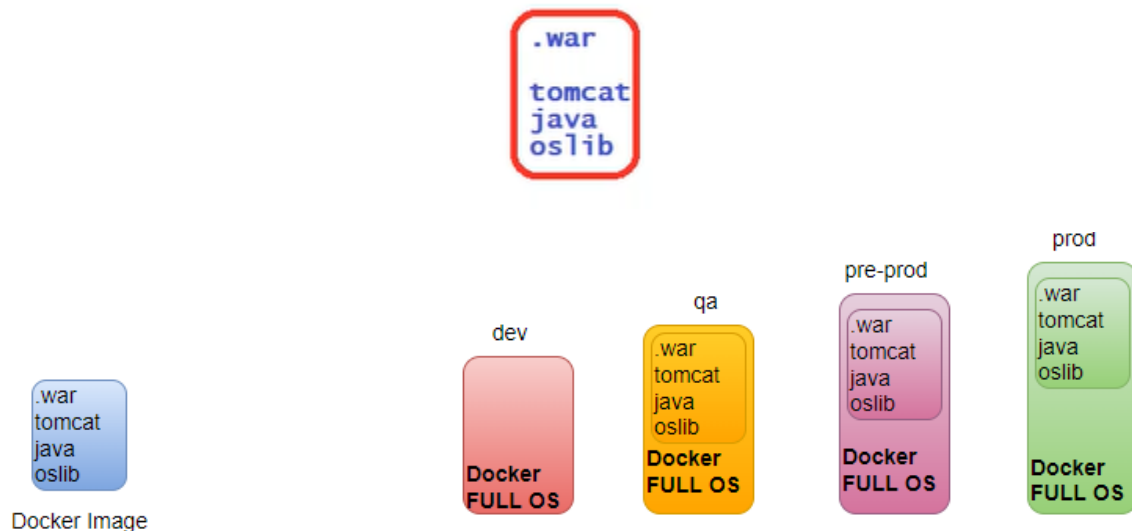
Problem: Environment (O/S, java, tomcat) already pre-configured before shipping, and in every release, we just shifting the artifact (.war). What about the dependencies? Cause may be mismatch in configuration (java level, O/S level, tomcat level) in various environment.

environment==a single /group of servers (**physical/virtual**) where we run our software/application



Solution? Container

Container: Is a package that consist of an app/software + all of its dependencies bundled together and deliver this package different environments



How to build and run the container?

What is the tool used to build .war (MAVEN), we convert the source code into a single executable file, similarly we need a tool to create a package (bundle artifacts and its dependencies in a single package) i.e., a containerization tool/container run time

- helps to build container package (Image)
- helps to run/deploy the containers

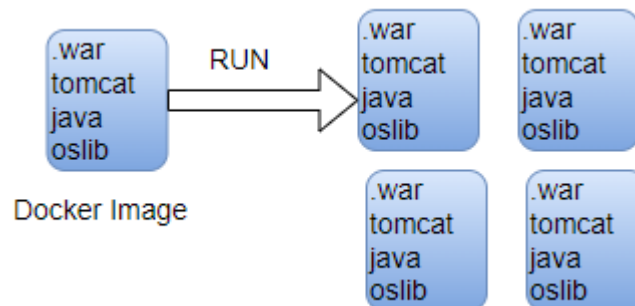
ex: docker/containerd/lxc/rocket/crio/podman

Docker is a tool help us BUILD the container->SHIP the container in different environments-> also RUN the package as a container.

Technically **package is called the docker image**, run time instance of the image is **container**. We can create multiple instances from a single image.

Docker Image:

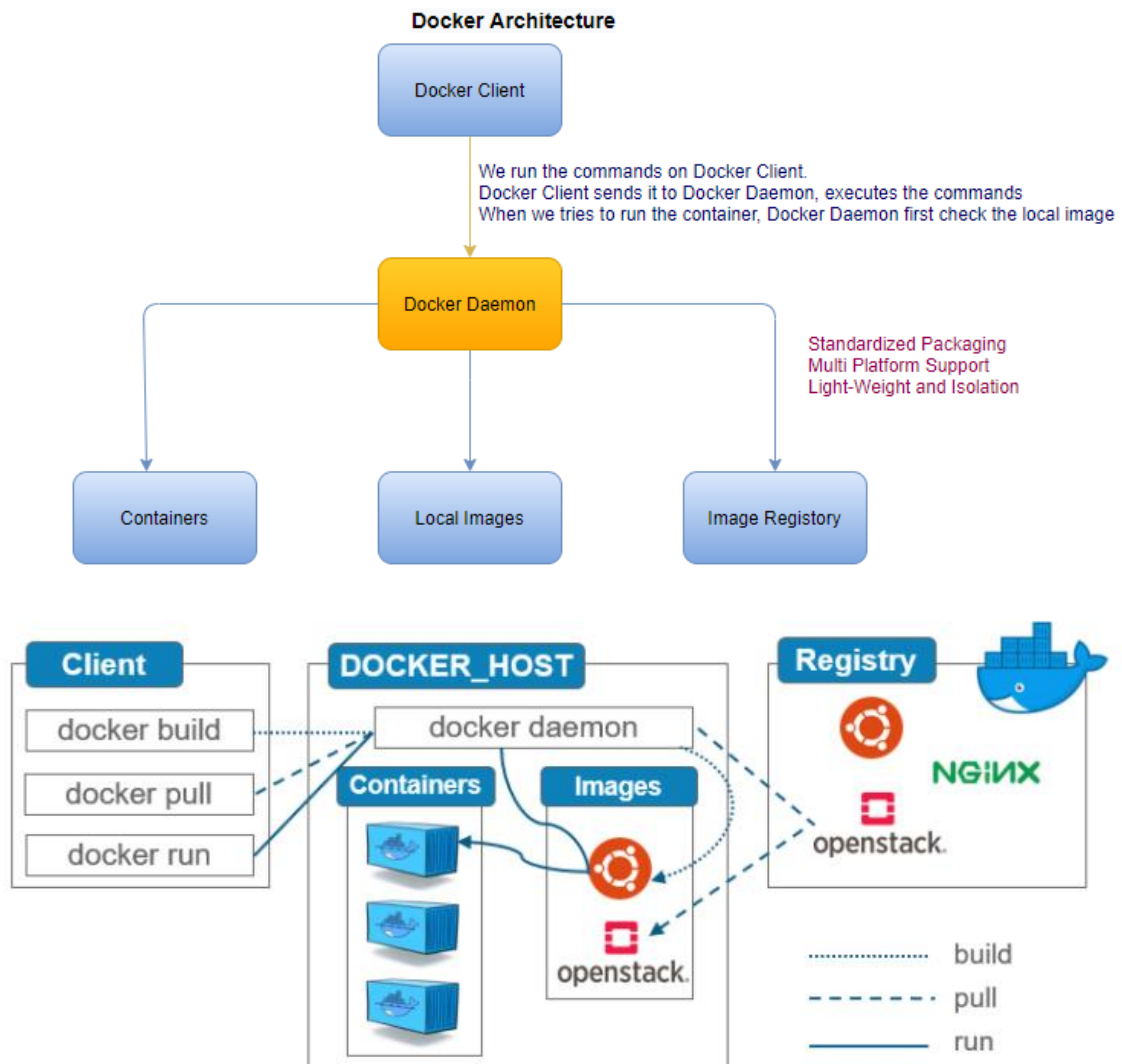
- Is a software package sitting on your disk using the disk space
- We copy the image/ install on server



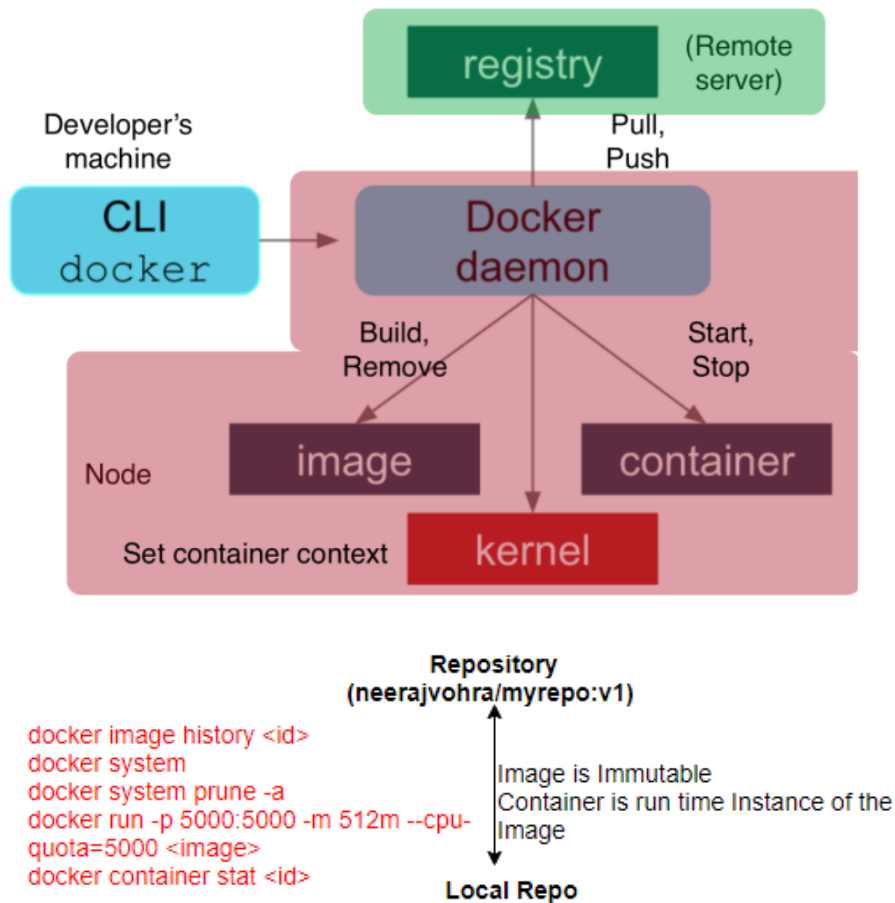
Docker Container:

- Container is runtime instance of an image.
- Container is just a process that run inside your computer/ server (physical/ vm) and consumes the resources (cpu/ram).
- From a single image we can run n number of containers
- Image is just like a car in parking mode (engine is in stop state). Image requirement **is just a storage** to store the File System just like car parking to store it filesystem, don't need to allocate any resources to the image, docker take care at run time Dynamic allocation of the resources.
- Container is like a car in running mode (engine is in running state use fuel and resources), container requirement is CPU and RAM (Memory).
- When you start the car now engine is in starting mode which solves the purpose of car-> container. When **car** starts it **consume petrol**, similar when **container** runs it consumes **some resources like CPU and RAM**.

Docker-Architecture



The docker client and daemon run in separate environments



How docker really works behind the scenes?

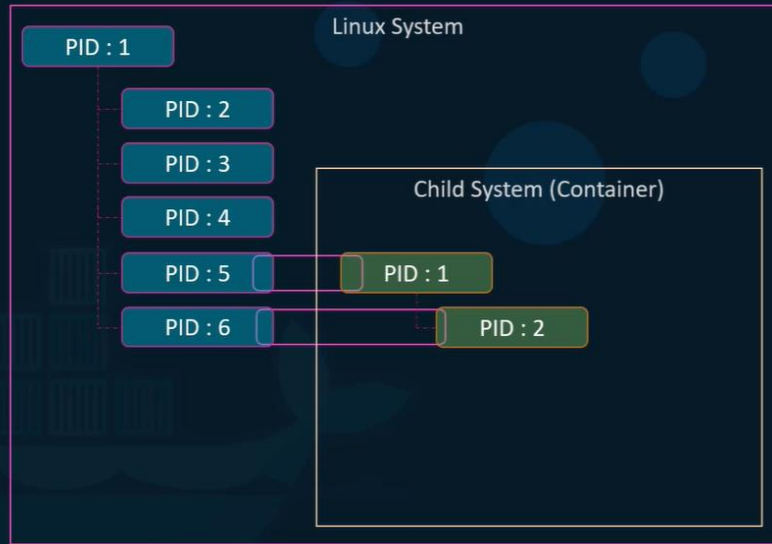
Where namespace, group and UnionFS is used in docker?

Namespace is responsible for **isolation** b/w the containers. Type of namespaces are PID, n/w, User, filesystem and so on (processId)

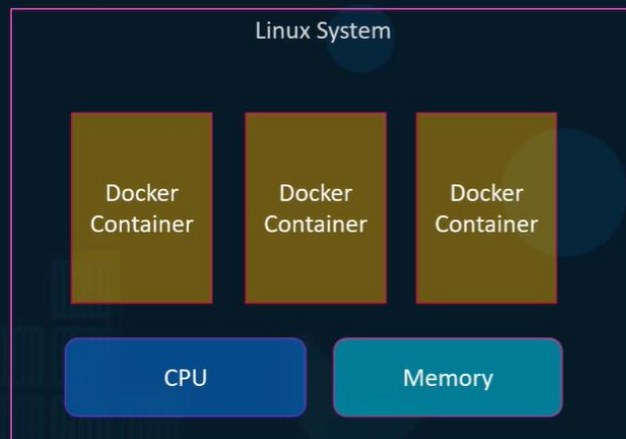
Cgroup is responsible for **resource management** or resource limitation in each container CPU, mem, Network and I/O.

UnionFS is a filesystem service which is used for creating **docker image layer**.

Namespace - PID



cgroups



```
docker run --cpus=.5 ubuntu
```

```
docker run --memory=100m ubuntu
```

`ps -eaf`

all the processes running inside the container

Is container replacement of VM?

No, container can be run on physical machine or on top of the VM.

If we have to run different-different applications then go for VM, if we have to execute multiple replicas of the same application then go for containers.

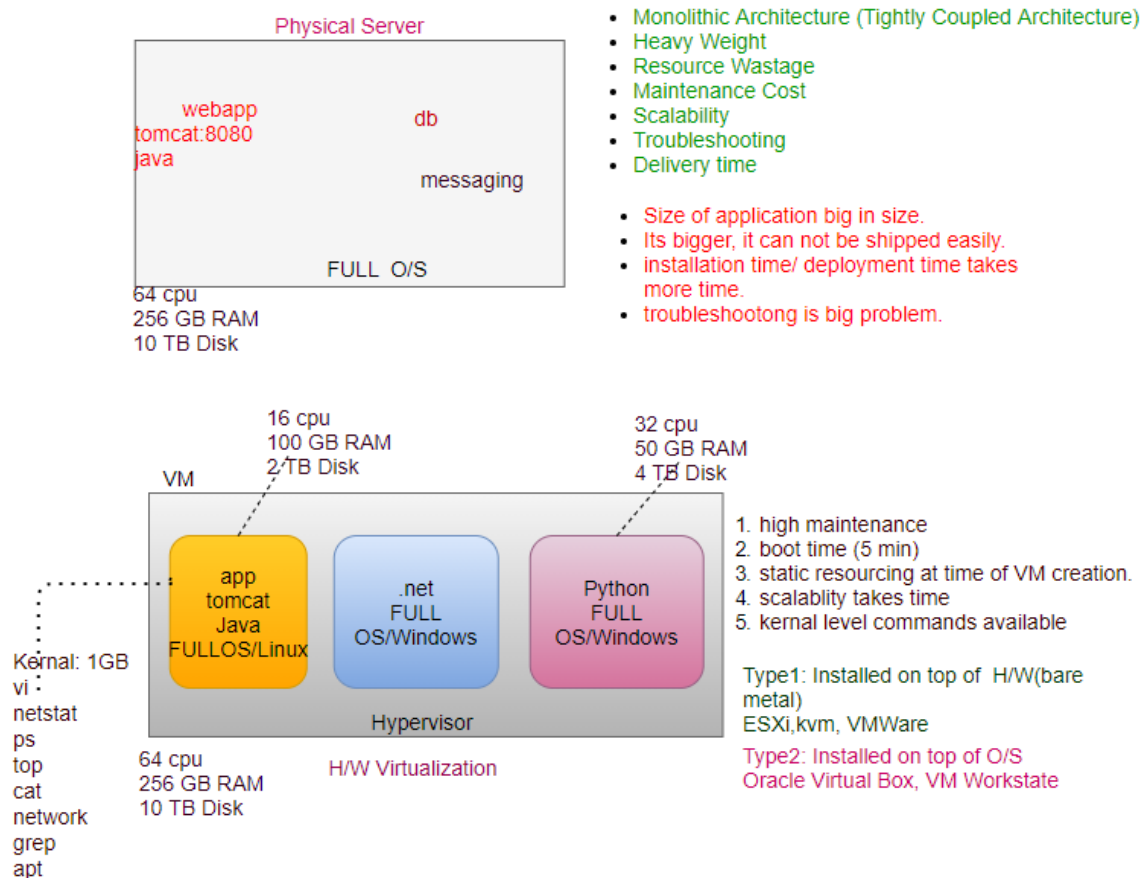
If your application is heavy then go with VM.

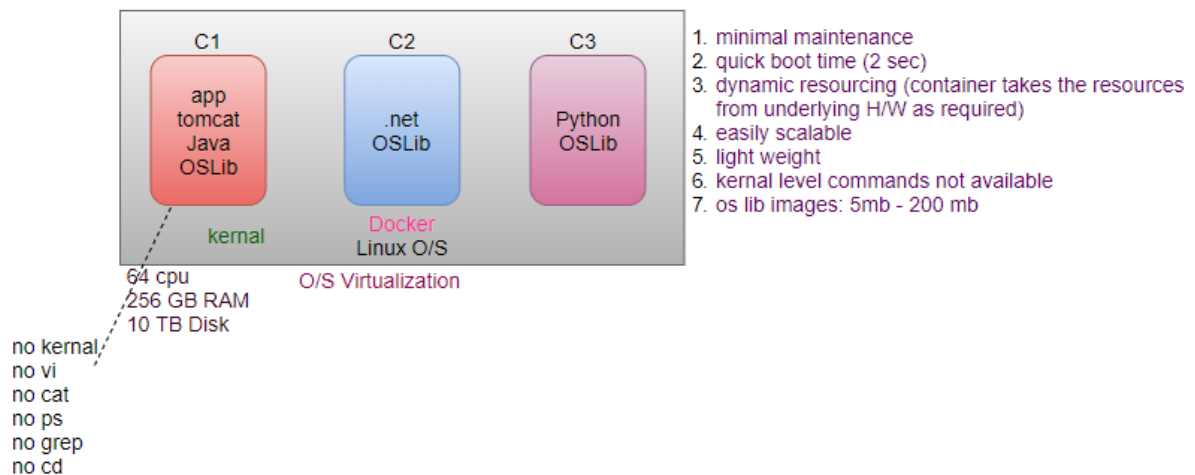
To Run the package in different environment we don't require any configuration on any environment, just docker installed on the environment no configuration required.

docker is open source by default - free docker-ce

docker also provides licensed s/w- paid docker-ee

Difference between Physical, VM and Container





In VM, we split the H/W resources in smaller pieces, Every VM provided a set of dedicated H/W resources. (H/W Virtualization, O/S level Isolation)

Every VM have full O/S.

Every VM have own IP address.

To run a container we need a machine, with a Full O/S either a physical machine or a VM, on top of that install Docker.

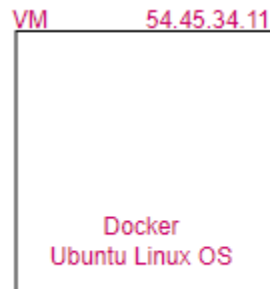
Every container has own IP address.

O/S Virtualization, App level Isolation

Difference between Full O/S and O/S lib (don't have dedicated kernel)?

- Minimum O/S (O/S lib) never requires license, they are open source and free.
- When we create a container, we don't allocate any resource to the container, because **container is just a process running on your system**, and process can take any **resources from underline H/W as required**. It's a **docker which provides necessary resources to the container dynamically**. Docker talks to the O/S kernal feature to provide the necessary resources to the container, then kernal talks to the H/W.

Installation



docker -v

/var/lib/docker

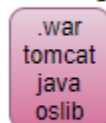
service docker status

To run a container what we need?

docker image: We will use some publicly available images from official docker image repository and run container from it.

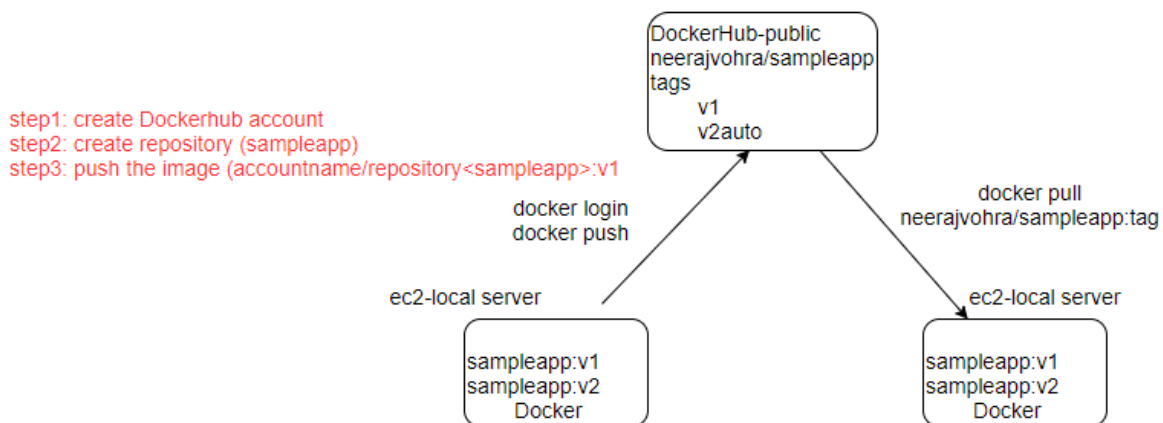
Once we understand about how to run container completely from given image then we will learn about how to build our own images.

tomcat image



Where we get the Images?

dockerhub to get the public images. Dockerhub has default account library.



`docker image ls` # to see if there is any image in system

`docker pull tomcat`

`docker pull nginx`

`docker image ls`

`/var/lib/docker`

`ls -l`

`docker image rm <ID>`

`cat /etc/docker/daemon.json` # default dockerhub

What is Docker Image?

- A docker image is a read-only template that contains source code, libraries, dependencies and set of instructions for start a container.
- docker image is immutable

What is Docker Container?

- container is running instance of the image.
- Every time you start a new container, an extra writable layer is created for it, where it stores the container specific changes.
- Changes are done on container, not on image (immutable)



COPY_ON_WRITE

How to run container from a given image?

To create/run a container from a given image we always use [docker run](#) command

Two Methods:

Interactive mode (-it)/ foreground

default

Whenever you want to interact with container O/S to validate what is inside? Or to make changes inside the container (dev use case)

`docker run image:tag`

`docker run -it tomcat:latest bash`

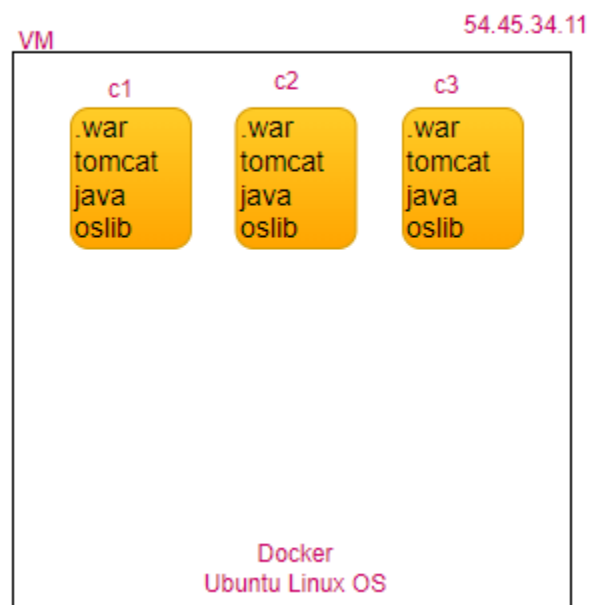
to come out of container always use `ctrl pq`

Container has a shell, if we have to interact with container we must have to go inside the container.e.g. We want to check the java version inside the container, or want to make some changes inside the container, in those cases we must have to interact with the container. Customization (on top of the base image).

detached mode(-d) / background

To run a container in background mode (production use case)

`docker run -d tomcat:latest`



How to know status of the container?

`docker container ls` # docker ps all running container

`docker container ls -a` # all containers running and dead

container is just like a **human body**, processes inside the containers are just like a **heartbeat** of human body if there is heartbeat in human body then its living body, without the heartbeat human body is dead body similarly if there is no process inside a container, container just goes into dead state (exit (0)).

`docker run -it <image> bash`

```
root@node2:~#  
root@node2:~# docker run -it tomcat:latest bash  
root@99f70d3f07e3:/usr/local/tomcat#  
root@99f70d3f07e3:/usr/local/tomcat#  
root@99f70d3f07e3:/usr/local/tomcat#
```

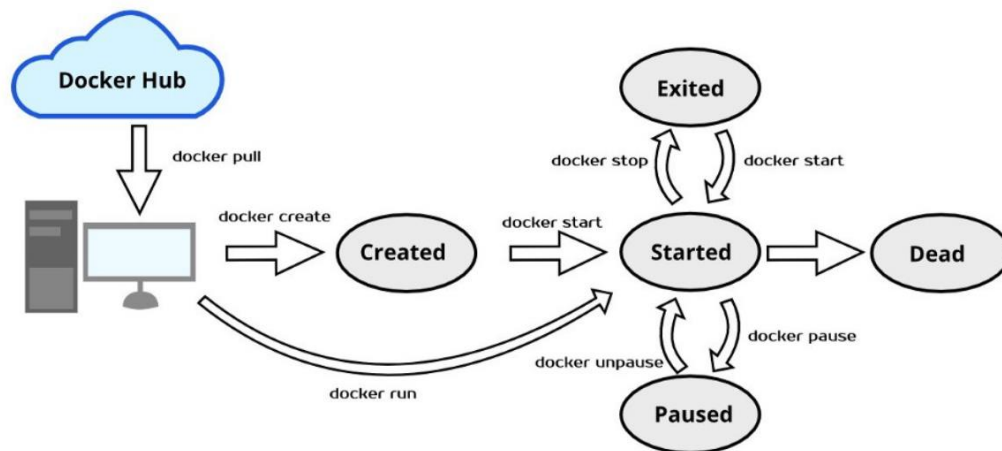
ctrl pq

How to get details of the container?

docker container inspect <container-id>

Container Lifecycle

- created
- running
- stop
- restart
- kill
- remove



First create an Image, create a container from Image, container may be in running state or in exited state. If in running state can be stopped, restarted, kill can be removed.

docker run nginx # foreground

docker run -d nginx # background

docker ps # shows all running containers on your server

docker stop <CONTAINERID>

`docker ps -a` # shows all running/stopped containers on your server

`docker restart <CONTAINERID>`

`docker kill <CONTAINERID>`

`docker rm -f <CONTAINERID>`

`docker stat` # CPU/Memory Utilization

`docker logs -f <CONTAINERID>` # follow live logs

`docker inspect <CONTAINERID>` # detailed information of container bridge network, dynamic IP

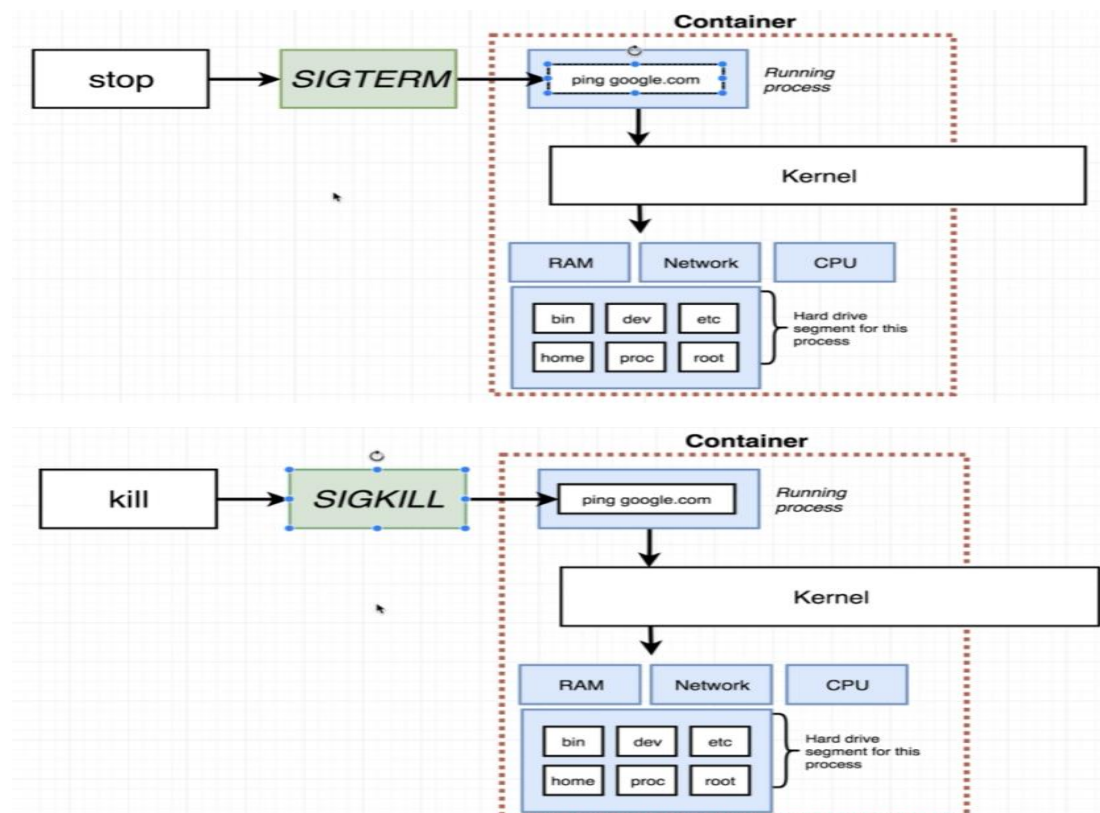
`/var/lib/docker/container`

`curl <IP>` # command line browser

`free`

`free -m`

Difference between docker stop and docker kill?



How to get inside the running container?

We can login to a container only if it is up, docker exec helps to login to a running container.

```
docker exec -it <containerId> bash
```

How to execute command inside the running container without getting inside?

```
docker exec <containerId> ls-l
```

```
docker exec <containerId> cat /etc/os-release
```

```
docker exec <containerid> java -version
```

```
docker exec <containerId> netstat
```

Will fail (O/S lib, default features of O/S not available)

Why you want to go inside the container?

Want to customized, want to know version of java running inside tomcat container etc.

How to access the application running inside the container from the outside?

Port Publish

How to publish the port?

```
docker run -d -P <image> # P dynamically allocate the port
```

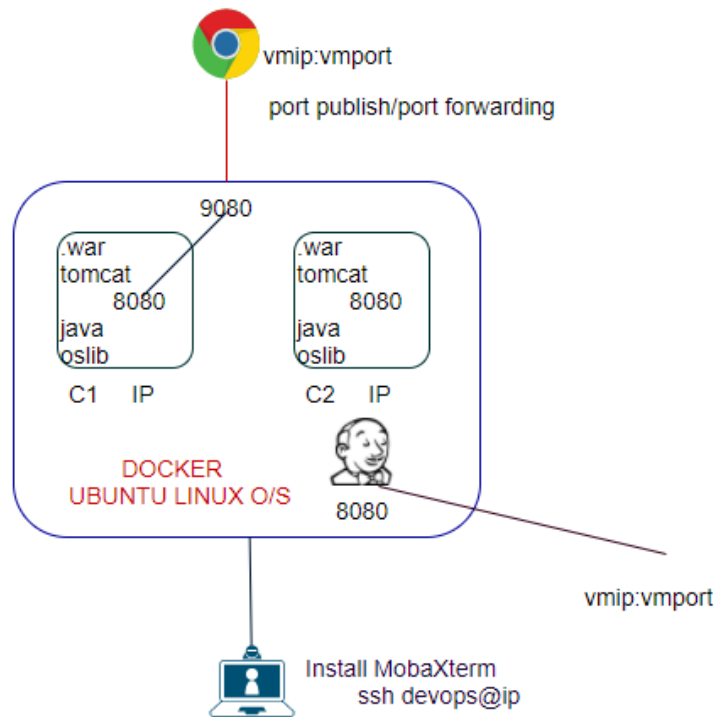
```
docker run -d -p 9090:8080 <image> #p <host>:<cont>allocate port statically
```

```
docker run -d -P lerndevops/samples: mywebsite-v1
```

containerip:containerPort

- can never be used in browser
- they internal only (can access only inside the server where the container is running)

browser: vmip(ec2ip):port



How to customize container name?

```
docker run -d --name <neeraj> <image>
```

How to rename container name?

```
docker rename <old name -clever_yonath> <new name- neeraj>
```

Advanced Container Commands

limit-container resources

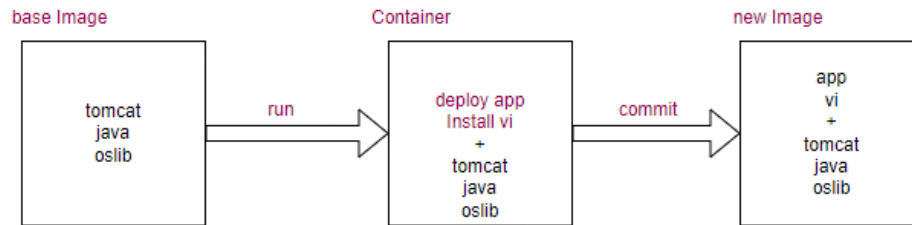
```
docker run -d -P --cpu=".5" nginx
```

```
docker run -d -P --memory 500M --memory-reservation 256M nginx
```

How to build own docker image?

- manual process
- automated process

Manual Process: creating an Image from container change



step1: take a base image (any image from Dockerhub)

```
docker pull tomcat:8.5.45
```

step2: run it as a container

```
docker run -d tomcat:8.5.45
```

step3: login to the container

```
docker exec -id <containerId> /bin/bash
```

step4: Inside the container make changes

```
/usr/local/tomcat
```

```
cd webapp
```

```
wget https://github.com/lerndevops/code/raw/main/sampleapp.war
```

```
#docker cp sampleapp.war container_id:/usr/local/tomcat/webapp/sampleapp.war
```

```
apt update
```

```
apt install -y vim
```

```
exit    # logout the container  ctrl pq
```

step5: Validate container still running

```
docker ps
```

step6: Create a new image

```
docker commit -m "any msg" <containerId> newimage:tag
```

step7: validate the newly created image

```
docker image ls
```

step8: Validate the Changes

```
docker run -d -P <Sampleapp:v1>
```

```

root@16d15c38db0f:/usr/local/tomcat/webapps# ls -l
total 4
drwxr-xr-x 2 root root 4096 Jan 30 15:16 myapp
root@16d15c38db0f:/usr/local/tomcat/webapps#
root@16d15c38db0f:/usr/local/tomcat/webapps# cd myapp/
root@16d15c38db0f:/usr/local/tomcat/webapps/myapp# ls -l
total 0
root@16d15c38db0f:/usr/local/tomcat/webapps/myapp# touch index.html
root@16d15c38db0f:/usr/local/tomcat/webapps/myapp# ls -l
total 0
-rw-r--r-- 1 root root 0 Jan 30 15:17 index.html
root@16d15c38db0f:/usr/local/tomcat/webapps/myapp# echo "My First Docker Image" > index.html
root@16d15c38db0f:/usr/local/tomcat/webapps/myapp# ls -l
total 4
-rw-r--r-- 1 root root 22 Jan 30 15:17 index.html
root@16d15c38db0f:/usr/local/tomcat/webapps/myapp#
root@16d15c38db0f:/usr/local/tomcat/webapps/myapp# read escape sequence
root@node1:~#
root@node1:~#
root@node1:~#
root@node1:~# docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS          NAMES
16d15c38db0f   tomcat    "catalina.sh run"        2 minutes ago Up 2 minutes   8080/tcp       amazing_greider
root@node1:~#
root@node1:~#
root@node1:~# docker commit -m "deployed myapp" 16d15c38db0f myapp:v1
sha256:37c654d807c59f36a8167ff07a3fe51e380af7a2e8a9f042538dac66a99916e4
root@node1:~#
root@node1:~#
root@node1:~# docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
myapp         v1        37c654d807c5   10 seconds ago 649MB
tomcat        latest    345867df0879   9 days ago    649MB
nginx         latest    f6d0b4767a6c   2 weeks ago   133MB

```

Automated Process

how to build a docker image ?

Automated Process — we will write a script called "Dockerfile"

- dockerfile is a plain text file without ext
- we will use some pre defined "dockerfile instruction" given by docker
ex: FROM / RUN / COPY etc..
- all dockerfile instructions are single words only
- dockerfile instruction words have to be written in capital letters always
- every line in a dockerfile must start with dockerfile instruction only

~~/~~ubuntu:18.04 — BASE IMAGE
~~/~~INSTALL JAVA 8
~~/~~INSTALL TOMCAT 9.X
~~/~~DEPLOY THE APP

vi Dockerfile (the file name can be anything)

FROM tomcat:8.5.45

RUN wget https://github.com/lerndevops/code/raw/main/myapp.war -P /usr/local/tomcat/webapps
 RUN apt-get update
 RUN apt-get install -y vim

how to execute the dockerfile script ?

docker build -f Dockerfile --tag myapp:v2 context (path-on-your-server)

nano Dockerfile

FROM tomcat:8.5.46 # pull the image, run as container, login into the container

RUN apt-get update

RUN apt-get install -y vim

RUN wget <https://github.com/lerndevops/code/raw/main/sampleapp.war> -P
 /usr/local/tomcat/webapps

docker build -f Dockerfile --tag sampleapp:v2 /root # in which directory Dockerfile available

docker image ls

docker run -d -P sampleapp:v2

UC2:

step1: download tomcat, jdk, myapp.war in root directory

step2: nano myapp

FROM ubuntu:18.04 # [pull the image, run as container, login into the container](#)

ADD instruction extracts/unzips the tar/zip files and copies the extracted contents to container

ADD jdk-8u331-linux-x64.tar.gz /tmp

RUN mv /tmp/jdk1.8.0_331 /opt/java

ADD apache-tomcat-9.0.63.tar.tz /tmp

RUN mv /tmp/apache-tomcat-9.0.63 /opt/tomcat

ENV JAVA_HOME /opt/java

ENV TOMCAT_HOME /opt/tomcat

ENV JAVA_VERSION 1.8

ENV TOMCAT_VERSION 9.0.63

COPY myapp.war /opt/tomcat/webapps/

ADD Instruction can also be used to download the file from given URL

RUN wget <https://github.com/lerndevops/code/raw/main/sampleapp.war> -P

#/usr/local/tomcat/webapps

#ADD <https://github.com/lerndevops/code/raw/main/sampleapp.war> /opt/tomcat/webapps

docker image inspect tomcat:9.0.63, sets the default port for the container that it helps docker to

#publish the ports when you use -P

EXPOSE 8080

The purpose of whole image creation is to start tomcat process when someone run the image

#helps to set default command for a image to execute when someone run image as container

CMD ["/opt/tomcat/bin/catalina.sh","run"]

docker build -f myapp --tag myapp:v1 /root

docker image ls

docker run -d -P myapp:v1

difference between ADD and COPY?

docker image ls

A Dockerfile is set of predefined instructions given by docker, which are used to construct a Docker Image. These instructions are called directives.

Instead of you run the instructions, docker executes the instructions on your behalf.

automate process: We will write a script called "Dockerfile"

- 1) Dockerfile is a plain text file without any extension
- 2) We will use some pre-defined Dockerfile instructions given by docker.
- 3) Dockerfile instruction is a single word and must be written in CAPITAL letters.
- 4) Every line in Dockerfile must start with Dockerfile instruction only:

vi Dockerfile

file name can be anything

```
FROM tomcat:8.5.40 # <accountid>/repository/tag
COPY target/sampleapp.war /usr/local/tomcat/webapps
EXPOSE 8080
CMD /usr/local/tomcat/bin/catalina.sh run
RUN touch /tmp/neeraj.txt
```

docker build -f Dockerfile --tag newimage:tag /root

Dockerfile directive:

FROM: start a new build stage and sets the base image, usually, must be the first directive in the Dockerfile (except ARG can be placed before FROM).

RUN: creates a new layer on top of the previous layer by running a command inside that new layer and committing the changes.

give instructions to the docker, docker runs these instructions within the container while preparing an image.

COPY: copy files from the local machine to the image.

ADD: same as COPY, additionally It can also copy files and directory from remote URL and also can copy the .tar file inside the container untar it in one shot.

CMD: default instruction Will be executed when container start from the given image, if someone wants to overwrite, the CMD instruction defined in the docker file, can be done at run time by passing another CMD.

We can pass it as an argument to the ENTRYPOINTS, CMD Instruction is a default argument to the ENTRYPOINT Instruction.

If there are more than 1 CMD then last will override previously CMD.

docker run <ubuntu> [command]

docker run ubuntu sleep 5 # default bash (process listen to terminal)

CMD ["sleep","5"] # first parameter must be executable

ENTRYPOINT: Is also run time instruction just like CMD instruction, we cannot overwrite the ENTRYPOINT instruction, if we trying at run time then newly command will be appended with the EntryPoint.

The ENTRYPOINT specifies a command that will always be executed when the container starts. The CMD specifies arguments that will be fed to the ENTRYPOINT.

Both CMD and ENTRYPOINT instructions define what command gets executed when running a container. There are few rules that describe their co-operation.

Dockerfile should specify at least one of CMD or ENTRYPOINT commands.

FROM Ubuntu

ENTRYPOINT ["sleep"]

In case of CMD instruction the command line parameter passed will gets replaced entirely, while in case of ENTRYPOINT command line parameters gets appended.

FROM Ubuntu

ENTRYPOINT ["sleep"]

CMD ["5"]

EXPOSE: 8080 documents which port(s) are intended to published when running a container. Docker understand container running on which port

ENV: set environment variables. These can be referenced in the Dockerfile itself and are visible to the container at runtime.

WORKDIR: sets the current working directory, you can also use relative path, which sets the new working directory relative to the previous working directory.

STOPSIGNAL: specify the signal that will be used to stop the container.

HEALTHCHECK: specify a command to run in order to perform a custom health check to verify that the container is working properly.

/*

<https://mirrors.estointernet.in/apache/tomcat/tomcat-9/v9.0.40/bin/apache-tomcat-9.0.40.tar.gz>

FROM ubuntu

RUN apt-get -y update

RUN apt-get -y install openjdk-11-jdk wget

RUN mkdir /opt/tomcat

ADD apache-tomcat-9.0.40.tar.gz /tmp

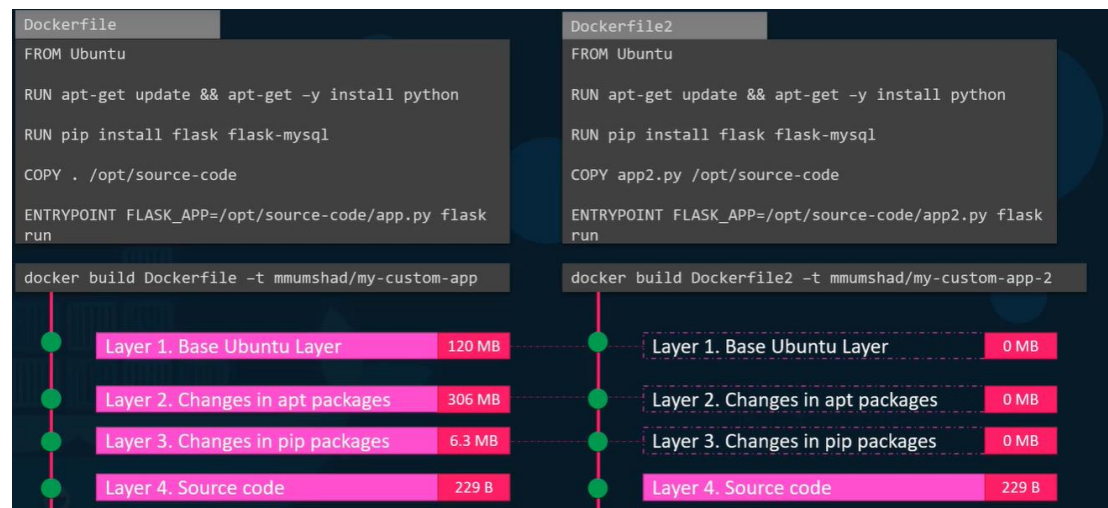
RUN mv /tmp/apache-tomcat-9.0.40/* /opt/tomcat/

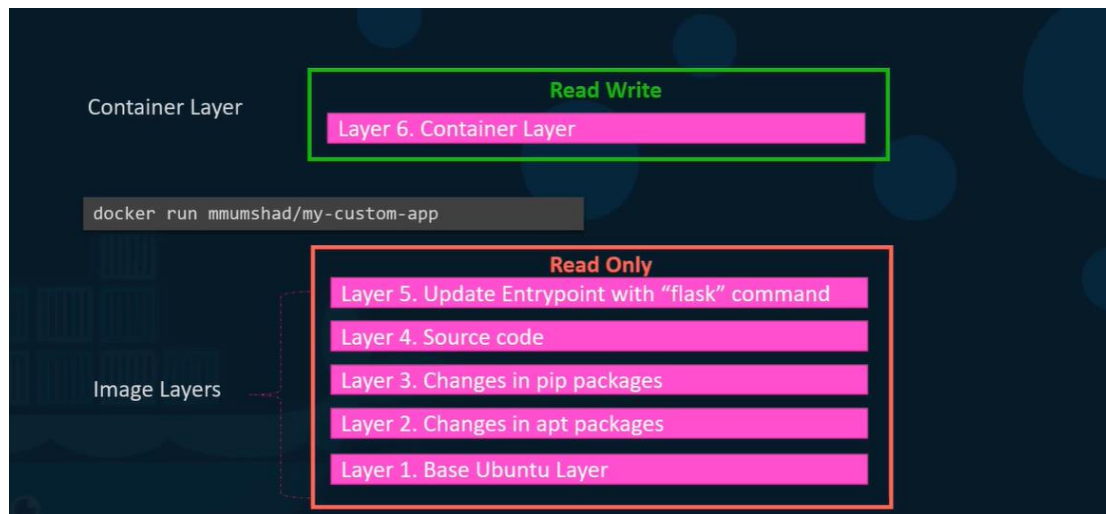
COPY myapp.war /opt/tomcat/webapps

EXPOSE 8080

ENTRYPOINT ["/opt/tomcat/bin/catalina.sh", "run"] ## JSON Format */

Layered Architecture





Each layer is a command, when its executed, creates intermediary image.

Docker Multistage Builds (Smaller Docker Image)

- Use Smaller base image
- Reduce the size of each layer
- Reduce number of layers

```
1 # set base image (host OS)
2 FROM python:3.8
3
4 # set the working directory in the container
5 WORKDIR /code
6
7 # copy the dependencies file to the working directory
8 COPY requirements.txt .
9
10 # install dependencies
11 RUN pip install -r ./requirements.txt
12
13 # copy the content to the working directory
14 COPY server.py .
15
16 # command to run on container start
17 CMD ["python", "./server.py"]
```



Layer 6 (CMD)
Layer 5 (COPY server.py .)
Layer 4 (RUN pip install)
Layer 3 (COPY requirements.txt)
Layer 2 (WORKDIR /code)
Layer 1 (FROM python)



Layer 6 (CMD)
Layer 5 (COPY server.py .)
Layer 4 (RUN pip install)
Layer 3 (COPY requirements.txt)
Layer 2 (WORKDIR /code)
Layer 1 (FROM python)

Writeable container layer
Layer 6 (CMD)
Layer 5 (COPY server.py .)
Layer 4 (RUN pip install)
Layer 3 (COPY requirements.txt)
Layer 2 (WORKDIR /code)
Layer 1 (FROM python)

```
1 # set base image (host OS)
2 FROM python:3.8
3
4 # set the working directory in the container
5 WORKDIR /code
6
7 # copy the dependencies file to the working directory
8 COPY requirements.txt .
9
10 # install dependencies
11 RUN pip install -r ./requirements.txt
12
13 # copy the content to the working directory
14 COPY server.py .
15
16 # command to run on container start
17 CMD [ "python", "./server.py" ]
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

1: powershell

```
PS C:\UdemyVideos\containers> docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
samplepyimage latest df06a79cd034 14 seconds ago 893MB
```

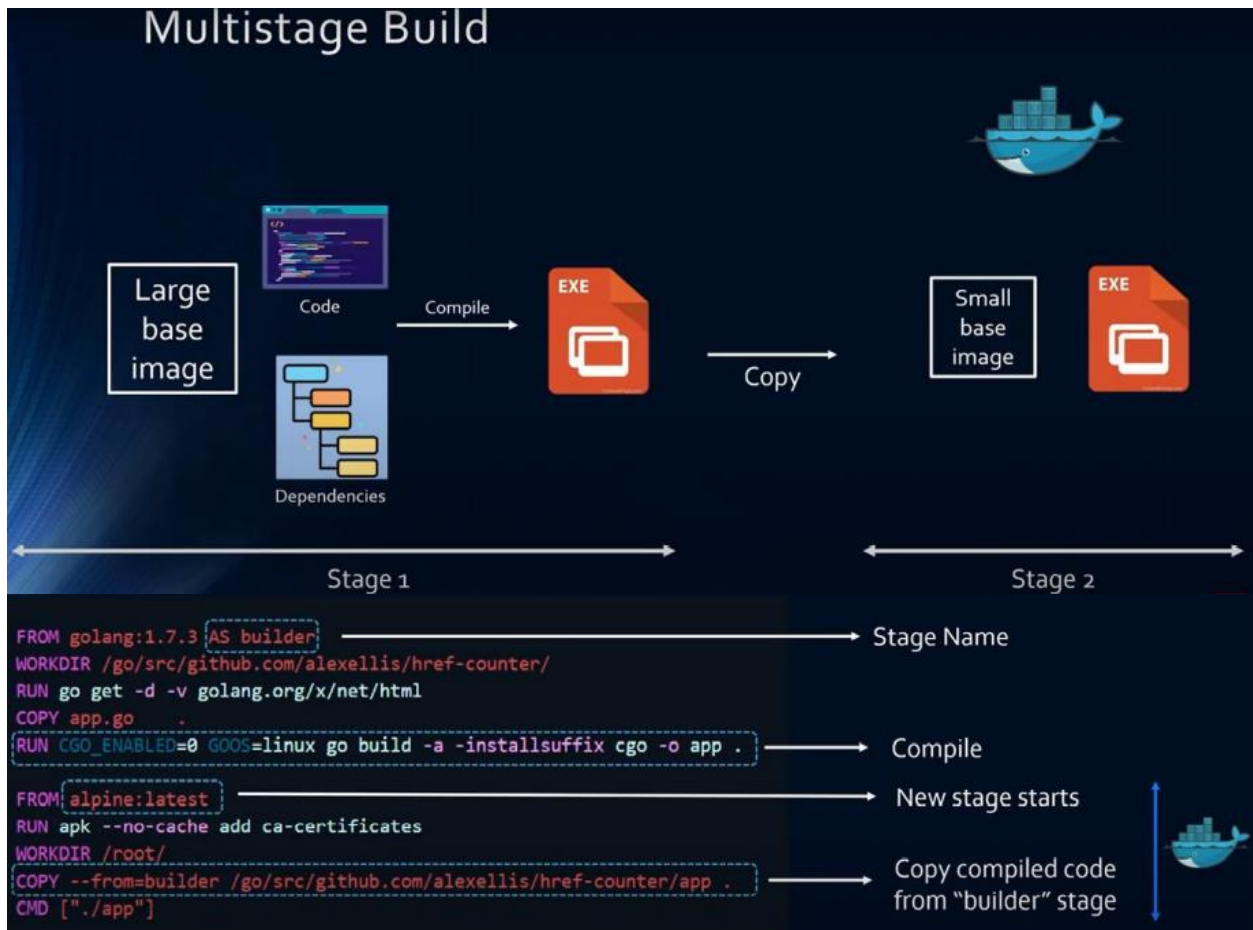
```
PS C:\UdemyVideos\containers> docker history samplepyimage
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
df06a79cd034	About a minute ago	CMD ["python" "./server.py"]	0B	buildkit.dockerfile.v0
<missing>	About a minute ago	COPY server.py . # buildkit	165B	buildkit.dockerfile.v0
<missing>	About a minute ago	RUN /bin/sh -c pip install -r ./requirements...	9.74MB	buildkit.dockerfile.v0
<missing>	About a minute ago	COPY requirements.txt . # buildkit	12B	buildkit.dockerfile.v0
<missing>	About a minute ago	WORKDIR /code	0B	buildkit.dockerfile.v0



Layer 1 (Base image)

FROM golang:1.7.3 → FROM alpine:latest



Who is responsible to maintain the layered architecture?

UnionFS is a filesystem service which is used for creating **docker image layer**.

Storage drivers

- AUFS
- ZFS
- BTRFS
- Device Mapper
- Overlay
- Overlay2

command in containers in Docker

UC1: you are running Ubuntu container

```
docker run ubuntu
docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
45aacca36850	ubuntu	"/bin/bash"	43 seconds ago	Exited (0) 41 seconds ago	

```
docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
45aacca36850	ubuntu	"/bin/bash"	43 seconds ago	Exited (0) 41 seconds ago	

Unlike a VM containers are not meant to host an O/S, containers are meant to host specific task or process, such as to host an Instance of the webserver, an instance of an application server or db server instance or to carry out some computation.



Once the task is over container exits. Container lives only till process inside it alive, if the webservice inside the container stops or crashes the container exits.

Who defines what process runs inside the container?

Ubuntu Image uses a bash as the default command, but bash is not really a process like a webserver or the database server, it's a shell that listens for input from a terminal , if it can not find the terminal it exits. When we ran the ubuntu container **docker run ubuntu** docker creates a container from ubuntu image and launch the bash program, by default docker doesn't attach a terminal to a container when its is run, so that bash program doesn't find a terminal so that it exits. Since the process that was started when the container was created finished the container exits as well.

How to specify a different command to start the container?

Option1: append the docker run command

It will overwrite the default command specified with in the image.

```
▶ docker run ubuntu [COMMAND]
```

```
▶ docker run ubuntu sleep 5
```

In this case when container will start, it will execute the sleep program and then exits.

UC: How to make this change permanent say you want image always run the sleep program when it starts?

You have to create your own image from the base ubuntu image and specify your own CMD

FROM Ubuntu

CMD sleep 5

CMD command param1

CMD ["command", "param1"]

CMD sleep 5

CMD ["sleep", "5"]

✓

CMD ["sleep 5"]

✗

```
▶ docker build -t ubuntu-sleeper .
```

```
▶ docker run ubuntu-sleeper
```



Remember whenever you create in JSON format first parameter must be executable.

UC: Want to change the number of seconds it sleeps currently its hard coded for 5 sec?

FROM Ubuntu

CMD sleep 5

```
▶ docker run ubuntu-sleeper sleep 10
```

Command at Startup: sleep 10

But we want to pass only the number of seconds, and sleep command must be invoked automatically

```
FROM Ubuntu
```

```
ENTRYPOINT ["sleep"]
```

```
docker run ubuntu-sleeper 10
```

ENTRYPOINT instruction is like a command instruction, you can specify the program that will be invoked when the container starts. Whatever you will specify in ENTRYPOINT will automatically appended when you run the container.

In case of CMD instruction the command line parameter passed will get replaced entirely, while in case of ENTRYPOINT command line parameters get appended.

UC: What will happen if in second case will run the command without append the number of seconds?

```
docker run ubuntu-sleeper
```

Then the command at the start-up will be **sleep** and we get the exception "missing operand"

UC: How to define the default value for the command line instruction?

You have to define ENTRYPOINT and CMD in JSON format

```
FROM ubuntu
```

```
ENTRYPOINT ["sleep"]
```

```
CMD ["5"]
```

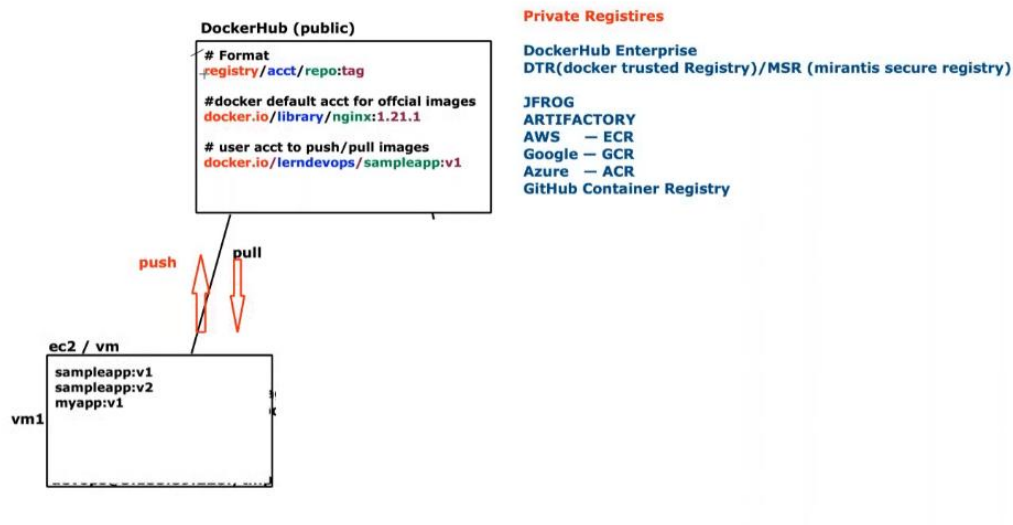
UC: How to overwrite the ENTRYPOINT instruction?

```
docker run --entrypoint sleep2.0 ubuntu-sleep
```

How to transfer images to another machine?

- Dockerhub -> push your image to remotely publicly accessible repository
- ECR -> Elastic Container Registry
- Offline

How to publish docker image to dockerhub/ How to share image available in local server?



```
docker pull gcr.io/google-containers/busybox:1.27
```

```
docker push # default [docker.io/library/repo]
```

```
docker build -f myapp --tag docker.io/neerajvohra/ learningjourney:myapp-v1 /root
```

```
# docker build -f Dockerfile --tag docker.io/neerajvohra/ learningjourney:myapp-v1 /root
```

```
docker image ls
```

```
docker login neerajvohra
```

```
# docker login gcr.io
```

```
# docker push neerajvohra/learningjourney:tagname
```

```
docker push docker.io/neerajvohra/learningjourney:myapp-v1
```

```
docker tag docker.io/library/tomcat:8.5.45 docker.io/neerajvohra/learningjourney:tomcat8.5.45
```

```
docker images # docker image ls
```

step1: Go to dockerhub create an account.

step2: Create a repository.

step3: come to your command line.

step4: `docker login` # default try for docker hub

```
# docker login jfrog.com/useraccount/repository
```

step5: Once successfully connected to the dockerhub.

step6: docker push sampleapp: v3 # denied

```
root@node1:~# docker push sampleapp:v3
The push refers to repository [docker.io/library/sampleapp]
eld7080edfbc: Preparing
eea3d21b5377: Preparing
07a7eb19e099: Preparing
8ae2de66e16b: Preparing
ca04b13b48c7: Preparing
b9eb87c9c542: Waiting
9f10818f1f96: Waiting
27502392e386: Waiting
c95d2191d777: Waiting
denied: requested access to the resource is denied
root@node1:~#
```

[docker.io/library (default docker account)/sampleapp]

step7: docker tag sampleapp:v3 <accountname>/<repository>: <image>

```
root@node1:~# docker tag sampleapp:v3 lerndevops/myappp:sampleapp-v3
root@node1:~#
root@node1:~# docker image ls
REPOSITORY          TAG                IMAGE ID           CREATED           SIZE
lerndevops/myappp   sampleapp-v3       1fa9c81f4ee3      14 minutes ago   555MB
sampleapp           v3                 1fa9c81f4ee3      14 minutes ago   555MB
sampleapp           v2                 13e5fc24be3b      18 minutes ago   555MB
sampleapp           v1                 04ea872b24a4      21 minutes ago   555MB
myapp               v2                 7dfb7e44e218      37 minutes ago   649MB
myapp               v1                 37c654d807c5      51 minutes ago   649MB
tomcat              latest            345867df0879      9 days ago       649MB
ubuntu              18.04            c090eaba6b94      9 days ago       63.3MB
nginx               latest            f6d0b4767a6c      2 weeks ago      133MB
root@node1:~#
root@node1:~#
root@node1:~# docker push lerndevops/myappp:sampleapp-v3
The push refers to repository [docker.io/lerndevops/myappp]
eld7080edfbc: Pushed
eea3d21b5377: Pushed
07a7eb19e099: Pushed
8ae2de66e16b: Pushed
ca04b13b48c7: Pushing [=====] 236.9MB/400.5MB
b9eb87c9c542: Pushed
c95d2191d777: Pushed
```

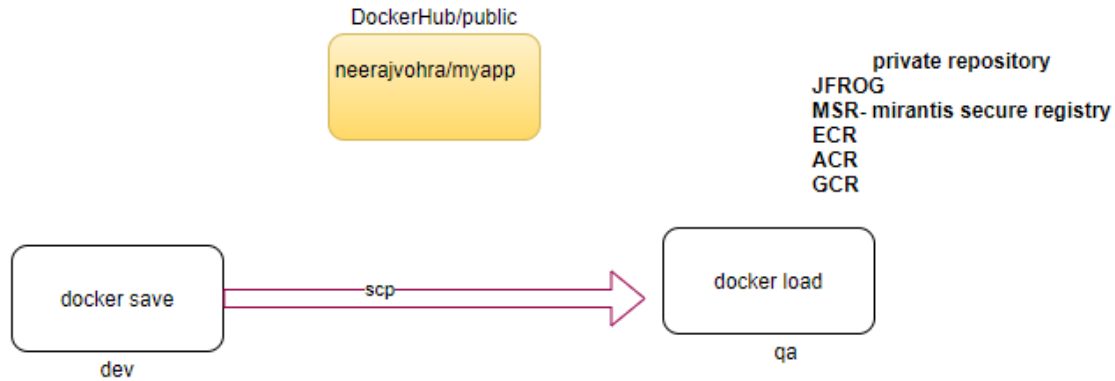
How to push image in private repository?

- Default remote is dockerhub (URL of JFrog)
- Default account is docker (Your Account)
- Default repository (Your Repository: Image)

docker tag <image name> <jFrog Id>/<account name>/<repository name>: tag

```
root@ip-172-31-21-138:~# docker images
REPOSITORY          TAG                IMAGE ID           CREATED           SIZE
lerndevops/myappp   v1                 952e59636e3a      21 hours ago     518MB
myapp               v1                 952e59636e3a      21 hours ago     518MB
myubuntu            v2                 598b7943fbc0      21 hours ago     210MB
myubuntu            v1                 e9649015beb9      22 hours ago     210MB
```

UC: How UC: transfer the image from one environment to another environment without remote repository.



step1: convert the image to .tar file (image is not a file its file system, we can't use scp command)

docker save -o myimages.tar.gz sampleapp: v3 sampleapp: v2 sampleapp: v2

```

root@node1:~# docker image ls
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
lerndevops/myapp    sampleapp-v3    1fa9c81f4ee3   20 minutes ago  555MB
sampleapp           v3             1fa9c81f4ee3   20 minutes ago  555MB
sampleapp           v2             13e5fc24be3b   25 minutes ago  555MB
sampleapp           v1             04ea872b24a4   28 minutes ago  555MB
myapp               v2             7dfb7e44e218   44 minutes ago  649MB
myapp               v1             37c654d807c5   58 minutes ago  649MB
tomcat              latest         345867df0879   9 days ago     649MB
ubuntu              18.04         c090eaba6b94   9 days ago     63.3MB
nginx               latest         f6d0b4767a6c   2 weeks ago    133MB
root@node1:~#
root@node1:~# docker save -o myimages.tar.gz sampleapp:v3 myapp:v2 myapp:v1
root@node1:~# ls -l
total 1218028
drwxr-xr-x 16 root root    4096 Jan  1 17:05 labs
-rw-r--r--  1 root root    190 Jan 30 15:31 mydfile
-rw-r--r--  1 root root 1230702592 Jan 30 16:19 myimages.tar.gz
-rw-r--r--  1 root root 16536021 Jan 30 15:42 sampleapp.war
-rw-r--r--  1 root root    377 Jan 30 15:56 sampleappdfile
root@node1:~#
  
```

step2: transfer the .tar file

scp myimages.tar.gz devops@10.128.0.18:/tmp

```

root@node1:~#
root@node1:~# scp myimages.tar.gz devops@10.128.0.18:/tmp
The authenticity of host '10.128.0.18 (10.128.0.18)' can't be established.
ECDSA key fingerprint is SHA256:/ij+RVuxnKns2xeb+aJuj1SmxmcD6EbiHT2orUrKZLI.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '10.128.0.18' (ECDSA) to the list of known hosts.
devops@10.128.0.18's password:
myimages.tar.gz
root@node1:~#
100% 1174MB 113.6MB/s 00:10
  
```

step3: login into target machine

cd /tmp

docker load < myimages.tar.gz


```

root@node2:~# docker image ls
REPOSITORY      TAG      IMAGE ID      CREATED      SIZE
root@node2:~# hostname -i
10.128.0.18
root@node2:~# cd /tmp
root@node2:/tmp# ls -l
total 1201880
-rw-r----- 1 devops devops 1230702592 Jan 30 16:21 myimages.tgz
drwx-r----- 2 devops devops      4096 Jan 30 14:56 ssh-avSaq9kegc
drwx-r----- 3 root   root      4096 Jan 30 14:56 systemd-private-dc55dabe36a64a6ea868525f49febca6-chrony.service-Noelap
drwx-r----- 3 root   root      4096 Jan 30 14:55 systemd-private-dc55dabe36a64a6ea868525f49febca6-systemd-resolved.service-qszsu8
drwx-r----- 2 root   root      4096 Jan 30 14:58 tmp6mwwu8en
root@node2:/tmp#
root@node2:/tmp# docker load < myimages.tgz
c95d2191d777: Loading layer [=====>] 65.62MB/65.62MB
27502392e386: Loading layer [=====>] 15.87kB/15.87kB
9f10818f1f96: Loading layer [=====>] 3.072kB/3.072kB
b9eb87c9c542: Loading layer [=====>] 35.16MB/35.16MB
ca04b13b48c7: Loading layer [=====>] 378.6MB/411.8MB

```